



**UNIVERSIDAD TECNOLÓGICA
DE QUERÉTARO**

Alumna:

Vasquez Noyola Andrea

Grupo:

IDGS17

Matrícula:

2023148002

Materia:

Seguridad en el Desarrollo de Aplicaciones

Profesor:

Gerardo Ramírez Villareal

Práctica 5

Red Docker Aislada

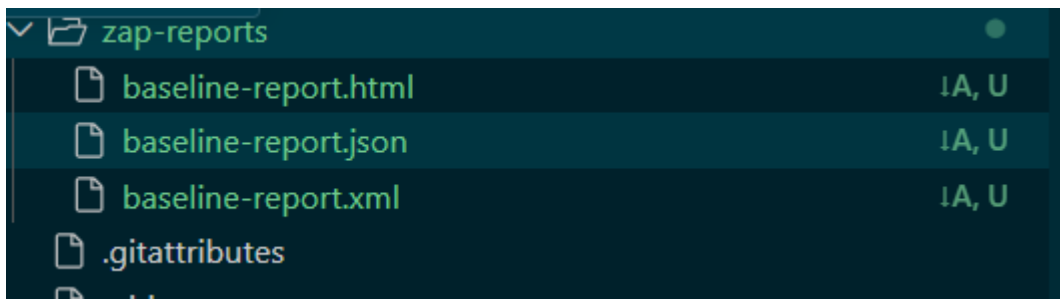
```
PS C:\Users\Andyy\OneDrive\Documentos\SDA_8vo\practicass\vulnerableapp>  
docker network inspect pentest-net
```

```
[  
  {  
    "Name": "pentest-net",  
    "Id": "ad5ab582f16349b2a7361220d488ca2ec0c630424f52c23edf289a66gbocf88b",  
    "Created": "2026-07-23T18:32:30.113861109Z",  
    "Scope": "local",  
    "Driver": "bridge",  
    "EnableIPv4": true,  
    "EnableIPv6": false,  
    "IPAM": {  
      "Driver": "default",  
      "Options": {},  
      "Config": [  
        {  
          "Subnet": "172.20.0.0/16",  
          "Gateway": "172.20.0.1"  
        }  
      ]  
    },  
    "Internal": true,  
    "Attachable": false,  
    "Ingress": false,  
    "ConfigFrom": {  
      "Network": ""  
    },  
    "ConfigOnly": false,  
    "Options": {  
      "com.docker.network.enable_ipv4": "true",  
      "com.docker.network.enable_ipv6": "false"  
    },  
    "Labels": {},  
    "Containers": {},  
    "Status": {  
      "IPAM": {  
        "Subnets": {  
          "172.20.0.0/16": {  
            "IPsInUse": 3,  
            "DynamicIPsAvailable": 65533  
          }  
        }  
      }  
    }  
  }  
]
```

Contenedores creados

```
PS C:\Users\Andyy\OneDrive\Documentos\SDA_8vo\practicass\vulnerableapp> docker compose -f pentest-compose.yml ps
time="2026-07-23T14:41:41-06:00" level=warning msg="project has been loaded without an explicit name from a symlink. Using name \"vulnerableapp\""
time="2026-07-23T14:41:41-06:00" level=warning msg="C:\\Users\\Andyy\\OneDrive\\Documentos\\SDA_8vo\\practicass\\vulnerableapp\\pentest-compose.yml: 'zap' is a reserved word"
Name                  Command                                                    Image              Port
vulnerable_app        vulnapp-vulnerable_app                                     dotnet/vulnerable_app 11 minutes ago    Up 11 minutes    8080/tcp
zap                    ghcr.io/zaproxy/zaproxy:stable                           sleep-infinity      zap                11 minutes ago    Up 11 minutes (unhealthy)
promtail               grafana/promtail:latest                                   /usr/bin/promtail -... 7 days ago        Up 7 days        1433/tcp
mssql                 mcr.microsoft.com/mssql/server:2022-latest              /opt/mssql/bin/laun... mssql              2 hours ago       Up 2 hours (healthy)
loki                  grafana/loki:latest                                       /usr/bin/loki -conf... loki                10 days ago       Up 10 days       0.0.0.0:3100->3100/tcp, [::]:3100->3100/tcp
```

zap/reports creados



Reporte ZAP

Archivo

C:\Users\Andyy\OneDrive\Documentos\SDA_8vo\practicass\vulnerableapp\zap-reports\baseline-report.html

ZAP by Checkmarx

ZAP Scanning Report

Site: http://vulnerable_app:8080

Generated on Thu, 23 Jul 2026 20:43:25

ZAP Version: 2.17.0

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	1
Informational	0
False Positives	0

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	2
Missing Anti-clickjacking Header	Medium	2
X-Content-Type-Options Header Missing	Low	2

Alert Detail

Alert Detail

Medium	Content Security Policy (CSP) Header Not Set
Description	Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.
URL	http://vulnerable_app.8080
Node Name	http://vulnerable_app.8080
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
URL	http://vulnerable_app.8080/
Node Name	http://vulnerable_app.8080/
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
Instances	2
Solution	Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html https://www.w3.org/TR/CSP/ https://w3c.github.io/webappsec-csp/ https://web.dev/articles/csp https://anusa.com/itsec/contentsecuritypolicy https://content-security-policy.com/
Reference	
CWE Id	693
WASC Id	15
Plugin Id	10038
Medium	Missing Anti-clickjacking Header
Description	The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.
URL	http://vulnerable_app.8080
Node Name	http://vulnerable_app.8080

Method	GET
Parameter	x-frame-options
Attack	
Evidence	
Other Info	
URL	http://vulnerable_app.8080/
Node Name	http://vulnerable_app.8080/
Method	GET
Parameter	x-frame-options
Attack	
Evidence	
Other Info	
Instances	2
Solution	Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app. If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options
CWE Id	1021
WASC Id	15
Plugin Id	10020
Low	X-Content-Type-Options Header Missing
Description	The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.
URL	http://vulnerable_app.8080
Node Name	http://vulnerable_app.8080
Method	GET
Parameter	x-content-type-options
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://vulnerable_app.8080/

URL	http://vulnerable_app.8080/
Node Name	http://vulnerable_app.8080/
Method	GET
Parameter	x-content-type-options
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
Instances	2
Solution	Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages. If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application/web server to not perform MIME-sniffing.
Reference	https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85) https://owasp.org/www-community/Security-Headers
CWE Id	693
WASC Id	15
Plugin Id	10021

Sequence Details

With the associated active scan results.

Tabla de Hallazgos

#	Hallazgo	Riesgo	CWE	CVSS estimado	URL afectada	Evidencia	Remediación
1	Broken Access Control — endpoint sin autenticación	High	CWE-862 (Missing Authorization)	7.5 — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N	GET /api/users	Respuesta 200 sin sesión activa, sin cookie, sin token — contrastar con GET /api/user/{id} que sí devuelve 401	Exigir sesión válida (y rol admin) antes de listar usuarios
2	Sensitive Data Exposure — contraseñas en texto plano en la respuesta	High (Crítico si se combina con #1)	CWE-200 + CWE-256	8.2 — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N (impacto de integridad por posible account takeover)	GET /api/users	Campo "password":"admin" visible en el JSON de respuesta	Proyectar solo campos no sensibles (Select(u => new { u.Id, u.Username, u.Email })); eliminar columna Password de texto plano, migrar 100% a PasswordHash con BCrypt
3	Absence of Anti-CSRF Tokens	Medium (ZAP: Low confidence)	CWE-352	4.3 — AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N	POST /Comment /AddComment	Formulario sin __RequestVerificationToken (confirmado por ZAP, pluginid 10202)	Agregar [ValidateAntiForgeryToken] + @Html.AntiForgeryToken() en la vista
4	Content Security Policy Header Not Set	Medium (High confidence)	CWE-693	5.3	Todo el sitio	Confirmado por ZAP (pluginid 10038) en /, /Search, /Comment	Middleware con Content-Security-Policy: default-src 'self'
5	Missing Anti-clickjacking Header	Medium	CWE-1021	4.3	Todo el sitio	Confirmado por ZAP (pluginid 10020)	X-Frame-Options: DENY

6	X-Content-Type-Options Header Missing	Low	CWE-693	3.1	Todo el sitio	Confirmado por ZAP (pluginid 10021)	X-Content-Type-Options: nosniff
---	---------------------------------------	-----	---------	-----	---------------	-------------------------------------	---------------------------------

1. **Nombre y tipo OWASP:** Sensitive Data Exposure combinado con Broken Access Control — mapea a OWASP API Security Top 10: API3:2023 (Broken Object Property Level Authorization) y A01:2021 (Broken Access Control).
2. **Request HTTP exacto:** GET http://vulnerable_app:8080/api/users — sin headers de autenticación, sin cookie de sesión.
3. **Response HTTP:** el JSON que pegaste arriba — status 200, con campo password en texto plano para los 3 usuarios.
4. **Por qué confirma la vulnerabilidad:** `_db.Users.ToList()` serializa el objeto User completo sin proyección (Select), y el método no valida `HttpContext.Session.GetInt32("UserId")` antes de ejecutar la consulta — a diferencia de `GetUser(int id)`, que sí lo hace línea por línea.
5. **Qué podría obtener un atacante real:** credenciales completas de todos los usuarios (username + password en claro) sin necesidad de autenticarse. Con eso puede iniciar sesión directamente vía `/Auth/Login` como cualquier usuario, incluido admin, y acceder al Dashboard con su Balance y datos personales — sin necesidad de explotar nada más.
6. **Código correcto:**

```
[HttpGet("users")]
public IActionResult GetAllUsers()
{
    var currentUserId = HttpContext.Session.GetInt32("UserId");
    if (!currentUserId.HasValue)
        return StatusCode(401, "No autenticado.");

    // TODO: agregar verificación de rol admin si tu modelo lo soporta

    var users = _db.Users
        .Select(u => new { u.Id, u.Username, u.Email }) // nunca
        .ToList();

    return Ok(users);
}
```